Viviane Flor Park
169259

Implementação de Compilador C-

São José dos Campos
2026

Viviane Flor Park
169259

Implementação de Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo

Instituto de Ciência e Tecnologia

São José dos Campos

2026

Lista de ilustrações

Figura 1 – Diagrama de blocos do processador. 7
Figura 2 – Visão geral da estrutura processador. 8
Figura 3 – Diagrama de blocos da fase de análise. 15
Figura 4 – Diagrama de atividades da análise léxica. 16
Figura 5 – Diagrama de atividades da análise sintática. 17
Figura 6 – Diagrama de atividades da verificação de símbolos da análise semântica. 18
Figura 7 – Diagrama de atividades da verificação de tipos da análise semântica. . 19
Figura 8 – Diagrama de blocos da fase de síntese. 26
Figura 9 – Diagrama de atividades da geração de código intermediário. 27
Figura 10 – Diagrama de atividades da geração de código assembly. 28
Figura 11 – Diagrama de atividades da geração de código binário. 28
Figura 12 – Modelo de gerenciamento de memória. 33
Figura 13 – Programa para cálculo de fatorial em `C-`. 34
Figura 14 – Programa para cálculo de maior divisor comum em `C-`. 37
Figura 15 – Programa para ordenação de vetor em `C-`. 41

Lista de tabelas

Tabela 1 – Instruções de tipo D. 11
Tabela 2 – Formato das instruções do tipo D. 12
Tabela 3 – Instruções de tipo B. 12
Tabela 4 – Formato das instruções do tipo B. 12
Tabela 5 – Instruções de tipo M. 13
Tabela 6 – Formato das instruções do tipo M. 13
Tabela 7 – Instruções de tipo A. 13
Tabela 8 – Formato das instruções do tipo A. 14
Tabela 9 – Convenções léxicas da linguagem `C-`. 20
Tabela 10 – Gramática da linguagem `C-`. 22
Tabela 11 – Formatos das quádruplas. 29
Tabela 12 – Formatos das instruções assembly. 30
Tabela 13 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de cálculo de fatorial. 34
Tabela 14 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de cálculo de maior divisor comum. 37
Tabela 15 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de ordenação de vetor. 41

Sumário

1 INTRODUÇÃO 6

2 PROCESSADOR 7
2.1 Visão Geral 7
2.2 Unidades Funcionais 8
2.2.1 *Program Counter* 8
2.2.2 Memória de Instruções e Unidade de Decodificação 8
2.2.3 Banco de Registradores 9
2.2.4 Unidade Lógica e Aritmética 9
2.2.5 Memória Principal 9
2.2.6 *Current Program Status Register* e Unidade de Validação 10
2.2.7 Unidade de Controle 10
2.2.8 Seletor de *Clock*, Entrada e Saída 10
2.3 Conjunto de Instruções 10
2.3.1 Instruções de Tipo D 11
2.3.2 Instruções de Tipo B 12
2.3.3 Instruções de Tipo M 12
2.3.4 Instruções de Tipo A 13
2.4 Organização da Memória 14

3 COMPILADOR: FASE DE ANÁLISE 15
3.1 Modelagem 15
3.2 Análise Léxica 19
3.3 Análise Sintática 21
3.4 Análise Semântica 23

4 COMPILADOR: FASE DE SÍNTESE 25
4.1 Modelagem 25
4.2 Geração de Código Intermediário 29
4.3 Geração de Código Assembly 30
4.4 Geração de Código Executável 31
4.5 Gerenciamento de Memória 32

5 EXEMPLOS 34
5.1 Fatorial Recursivo 34
5.2 Maior Divisor Comum 37

5.3 ***Sort*** **40**

6 **CONCLUSÃO** **50**

REFERÊNCIAS **51**

1 Introdução

Em meados do século XX, surgiram os primeiros computadores, primeiramente destinados a usos científicos e militares. Estes eram programados em linguagem de máquina, ou seja, de baixo nível, o que tornava o simples uso de um computador uma tarefa trabalhosa, muito sujeita a erros, e que exigia conhecimento aprofundado da máquina e de seu conjunto de instruções.

A evolução dos sistemas computacionais criou uma demanda por computadores mais rápidos e com maior capacidade de processamento. A partir desta demanda, foram criadas linguagens de programação de mais alto nível, capazes de representar algoritmos de forma mais eficiente e próxima da linguagem humana e da lógica matemática. Naturalmente, surgiu também a necessidade de um mecanismo que traduzisse esses programas para código de máquina. Foi nesse contexto que nasceram os compiladores, ferramentas responsáveis por transformar o código-fonte escrito pelo programador em um programa executável pelo computador.

Além de automatizar esse processo de tradução, os compiladores desempenham um papel fundamental na evolução da computação moderna. Eles permitem que um mesmo algoritmo seja desenvolvido de maneira mais legível, segura e produtiva, abstraindo detalhes específicos da arquitetura do processador e facilitando a portabilidade entre diferentes plataformas. Durante a compilação, também são realizadas diversas análises e otimizações que detectam erros, melhoram o desempenho do código gerado e reduzem o consumo de recursos computacionais.

Este projeto, desenvolvido durante a disciplina de Laboratório de Sistemas Computacionais: Compiladores do ICT-Unifesp, visa implementar um compilador para a linguagem de programação `C-` (LOUDEN, 2004), de forma que gere código de máquina de acordo com um processador ARM 32 bits (PARK, 2025) desenvolvido durante a disciplina anterior de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores da mesma instituição. Para tal, este relatório divide-se em três principais estágios de desenvolvimento do projeto: descrição do processador e seu conjunto de instruções, feita na seção 2; descrição da fase de análise do compilador, onde são comentadas as análises léxica, sintática e semântica, feito na seção 3; e descrição da fase de síntese do compilador, onde é detalhado o processo de geração de código, do intermediário ao binário, feito na seção 4.

2 Processador

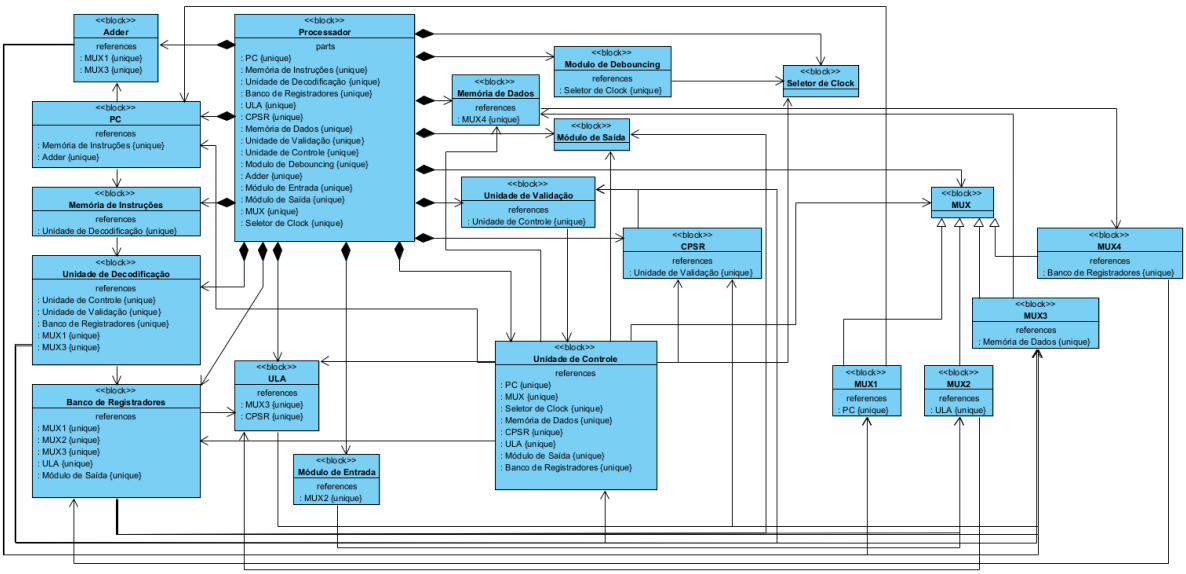
Nesta seção, é descrito o processador utilizado para o desenvolvimento do projeto do compilador `C-`.

2.1 Visão Geral

O processador utilizado para o desenvolvimento deste projeto implementa uma arquitetura ARM monociclo adaptada de 32 bits (PARK, 2025). Ele conta com 32 registradores de propósito geral, entre os quais o registrador R31 foi reservado para uso como *link register* (LR). Também foram projetados dois registradores de uso dedicado: o *program counter* (PC) e o *current program status register* (CPSR), excepcionalmente de 4 bits. O processador também conta com um sistema de interface com um *field programmable gate array* (FPGA).

O diagrama de blocos do processador, observado na Figura 2, mostra todas as diferentes unidades funcionais que o compõem e suas interações. Detalhes a respeito dos propósitos específicos, funcionamento e interligação dessas unidades são descritos na seção 2.2.

Figura 1 – Diagrama de blocos do processador.



Fonte: A autora (2026).

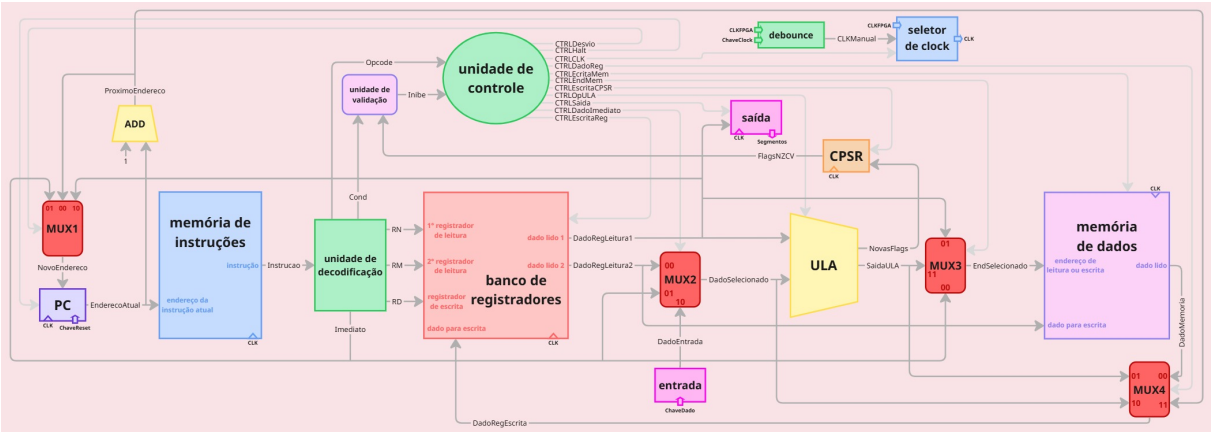
Fazem parte do processador o PC, memória de instruções, unidade de decodificação, banco de registradores, unidade lógica e aritmética (ULA), CPSR, memória de dados, unidade de validação, unidade de controle, multiplexadores e *adder*. Existem quatro

instâncias de multiplexadores no projeto, cada qual controla o fluxo de dados em um lugar diferente do *datapath*, observado na Figura 2. Estão presentes no diagrama também os módulos do sistema de interface com FPGA do processador, como os módulos de entrada e saída, seletor de *clock* e unidade de *debouncing*.

2.2 Unidades Funcionais

Para uma visualização mais clara da maneira como as diferentes unidades funcionais do processador se interligam e como ocorre o fluxo de dados, é apresentado na Figura 2 uma visão geral da estrutura do processador.

Figura 2 – Visão geral da estrutura processador.



Fonte: (PARK, 2025).

2.2.1 *Program Counter*

O PC é um registrador especial que guarda o endereço da instrução executada. A cada borda de subida do *clock*, essa informação é atualizada seja pelo incremento unitário do *adder*, por um endereço contido em um registrador qualquer do banco de registradores ou por um endereço de desvio contido em uma instrução. Essa decisão é feita por um multiplexador (nesse caso, o MUX1), conforme visto na Figura 2, com o auxílio do sinal CTRLDesvio advindo da unidade de controle.

A atualização desse registrador pode ser inibida pelo sinal de controle CTRLHalt, emitido pela unidade de controle diante de uma instrução HLT, conforme explicitado na seção 2.3. Além disso, seu conteúdo pode ser resetado por meio da entrada ChaveReset, que foi implementada como uma chave de uma placa FPGA.

2.2.2 Memória de Instruções e Unidade de Decodificação

O conteúdo do PC é alimentado à memória de instruções, que lê a instrução contida naquele endereço e a disponibiliza em sua saída. Esse processo é realizada de forma

síncrona, com a borda de descida do *clock*.

A instrução, então, é fornecida à unidade de decodificação, que identifica o seu tipo, conforme descrito na seção 2.3, e a particiona nos campos correspondentes.

2.2.3 Banco de Registradores

O banco de registradores recebe dois endereços de leitura e um de escrita, além do dado para a escrita. A leitura dos registradores é feita de forma assíncrona e disponibilizada prontamente nas saídas do banco, conforme visto na Figura 2. A escrita em registradores, por outro lado, é feita de forma síncrona com a borda de subida do *clock* de acordo com o sinal de controle CTRLEscritaReg.

Os dados lidos do banco de registradores são disponibilizados para diversas outras unidades funcionais do processador, como para o próprio PC, conforme já descrito anteriormente, para a ULA, memória de dados ou módulo de saída. Esses dados podem até ser realimentados para o próprio banco.

2.2.4 Unidade Lógica e Aritmética

A ULA tem duas funções principais: realizar operações lógicas e aritméticas e, com base nestas, fazer o cálculo das novas *flags* NZCV do CPSR, conforme descrito na seção 2.2.6. Ela recebe dois operandos, que podem vir do banco de registradores ou da instrução, além do sinal CTRLOpULA, vindo da unidade de controle, que decide qual operação será realizada sobre os dados. A ULA pode realizar 9 operações distintas: soma, subtração (de duas maneiras diferentes, invertendo a ordem dos operandos), multiplicação, divisão, *not*, *and*, *or* e *xor*.

Os resultados da operações realizadas por ela podem ser alimentados à memória principal ou ao banco de registradores. As *flags* calculadas são sempre fornecidas ao CPSR.

2.2.5 Memória Principal

A memória de dados recebe um endereço que pode ser tanto de leitura, quanto de escrita. O conteúdo contido nesse endereço sempre é disponibilizado em sua saída de maneira assíncrona. Já a escrita de um dado neste endereço é feita de forma síncrona com a borda de subida do *clock*, de acordo com o sinal de controle CTRLEscritaMem.

O endereço fornecido à memória pode ter origem na instrução, no caso de endereçamento direto, no banco de registradores, no caso de endereçamento indireto por registrador, ou no resultado de uma operação da ULA, para o endereçamento por deslocamento. A seção 2.3.3 discute mais sobre as diferentes formas de endereçamento à memória às quais o processador oferece suporte.

2.2.6 *Current Program Status Register* e Unidade de Validação

O CPSR e a unidade de validação são, juntas, o coração da execução condicional de instruções, que é o diferencial da arquitetura ARM.

O CPSR é um registrador especial que guarda as *flags* NZCV, quatro bits de informação que dizem respeito ao valor do resultado da ULA. Essa informação consiste nas *flags* N (*negative*), que indica se o resultado é negativo, Z (*zero*), que indica se o resultado é zero, C (*carry*), que indica se o resultado gerou um bit de *carry*, e V (*overflow*), que indica se o resultado estourou seu tamanho máximo.

Conforme visto na Figura 2, as informações do CPSR são alimentadas à unidade de validação junto a um campo especial da instrução, chamado de *cond*. Essas informações são comparadas e, então, é gerado um sinal que decide se a execução da instrução deve ou não ser inibida. A correspondência entre essas informações é discutida com mais detalhes na seção 2.3.

2.2.7 Unidade de Controle

A unidade de controle recebe o *opcode* da instrução e decide, então, a configuração dos sinais de controle. Além disso, caso a unidade de validação tenha gerado um sinal positivo de inibição, ela trata a instrução atual como uma instrução NOP.

2.2.8 Seletor de *Clock*, Entrada e Saída

O seletor de *clock* e os módulos de entrada e saída fazem parte do sistema de interface com FPGA do processador. O primeiro realiza a divisão de frequência do *clock* interno de 50MHz do FPGA, tornando a frequência do *clock* padrão que rege o processamento 1MHz, além de ter o poder de alterar esse *clock* para uma chave da placa, de forma que funcione de maneira manual. Essa segunda funcionalidade do seletor de *clock* é utilizada em instruções IN e OUT.

O módulo de entrada lê um dado codificado em binário nas chaves do FPGA e o disponibiliza para processamento posterior. Ele é utilizado em instruções IN, que guardam essa informação em um registrador. Já o módulo de saída recebe a informação em binário de um registrador e faz sua conversão de modo que possa ser visualizado nos *displays* de 7 segmentos do FPGA.

2.3 Conjunto de Instruções

O conjunto de instruções ao qual o processador oferece suporte pode ser subdividido em quatro categorias: instruções de processamento de dados ou tipo D, de desvio ou tipo B, de acesso à memória ou tipo M e alternativas ou tipo A.

Todas as instruções têm os 6 bits mais significativos em comum. Os quatro primeiros compõem o campo *cond* da instrução, que impõe uma condição de execução sobre ela, e os dois bits seguintes, que compõem o campo op0 e fazem parte do *opcode*, servem para classificar o tipo da instrução. Essa classificação é importante porque, conforme descrito a seguir, os formatos das instruções variam de acordo com o seu tipo.

2.3.1 Instruções de Tipo D

As instruções de processamento de dados, ou de tipo D, realizam operações aritméticas, lógicas, de movimentação ou de comparação entre valores. Essas instruções estão listadas na Tabela 1.

Vale ressaltar que somente algumas instruções podem alterar as *flags* do CPSR, como as instruções ADDS, SUBS, RSB e CMP. Além disso, todas as instruções de tipo D têm suporte para imediatos no lugar do registrador RM.

Tabela 1 – Instruções de tipo D.

Tipo	**Mnemônico**	**op1**	**Descrição**
Aritméticas	ADD	00000	RD = RN + RM/Im
	ADDS	00001	RD = RN + RM/Im; Altera *flags*
	SUB	00010	RD = RN - RM/Im
	SUBS	00011	RD = RN - RM/Im; Altera *flags*
	RSB	00100	RD = RM/Im - RN; Altera *flags*
	MUL	00101	RD = RN · RM/Im
	UDIV	00110	RD = RN ÷ RM/Im
Lógicas	NOT	00111	RD = RM/Im
	AND	01000	RD = RN and RM/Im
	ORR	01001	RD = RN or RM/Im
	EOR	01010	RD = RN xor RM/Im
Movimentação	MOV	01011	RN = RM/Im
Comparação	CMP	01100	RN - RM/Im; Altera *flags*

Fonte: (PARK, 2025)

A Tabela 2 mostra o formato das instruções de tipo D. O *opcode* tem tamanho de 8 bits e é dividido em op0, bit I e campo op1. Este último é próprio de cada instrução e é utilizado para identificar a operação realizada. Ele pode ser observado na Tabela 1 junto a cada instrução. Já o bit I é utilizado para identificar se a instrução deve interpretar a informação nos bits 9 a 0 como um registrador ou valor imediato.

Tabela 2 – Formato das instruções do tipo D.

Formato	***cond***	**op0 (00)**	**I**	**op1**	**RN**	**RD**	**RM/Imediato**
Bits	31:28	27:26	25	24:20	19:15	14:10	9:0

Fonte: (PARK, 2025)

2.3.2 Instruções de Tipo B

As instruções de desvio ou *branch*, ou de tipo B, realizam saltos. A lógica da execução condicional de instruções também se aplica a elas, apesar de, a princípio, operarem de forma incondicional. Essas instruções estão listadas na Tabela 3.

Tabela 3 – Instruções de tipo B.

Mnemônico	**op1**	**Descrição**
B	00	PC = Endereço
BX	01	PC = RN
BL	10	LR = PC + 1; PC = Endereço

Fonte: (PARK, 2025)

A Tabela 4 mostra o formato das instruções de tipo B. O *opcode* tem tamanho de 4 bits e é dividido nos campos op0 e op1. Cada instrução possui uma configuração única dos bits em op1, conforme visto na Tabela 3, que indica como deve ser executada. Os bits 18 a 0 contêm um endereço absoluto de desvio, e é particularmente relevante nas instruções B e BL.

Tabela 4 – Formato das instruções do tipo B.

Formato	***cond***	**op0 (01)**	**op1**	**RN**	**Endereço**
Bits	31:28	27:26	25:24	23:19	18:0

Fonte: (PARK, 2025)

2.3.3 Instruções de Tipo M

Instruções de acesso à memória, ou de tipo M, carregam dados da memória principal em registradores e vice-versa. Existem duas instruções de tipo M: LDR (*load*) e STR (*store*). Elas podem ser configuradas de modo que acessem a memória de diferentes maneiras, conforme listado na Tabela 5.

Tabela 5 – Instruções de tipo M.

Tipo	**Endereçamento**	**bits S, I e R**	**Descrição**
Load	Imediato	010	RD = Imediato
	Direto	000	RD = (Endereço)
	Indireto	001	RD = ((RN))
	Deslocamento	011	RD = ((RN + *offset* imediato))
Store	Direto	100	Endereço = RM
	Indireto	101	(RN) = RM
	Deslocamento	111	(RN + *offset* imediato) = RM

Fonte: (PARK, 2025)

A Tabela 6 mostra o formato das instruções de tipo M. O *opcode* tem tamanho de 5 bits e é composto pelo campo op0 e pelos bits S, I e R. Esses bits indicam, repectivamente, se a instrução é de *load* ou *store*, se os bits 12 a 0 são irrelevantes ou se contêm um endereço ou *offset* imediato, e se os bits 22 a 18 contêm um registrador. O bit I é setado para instruções que endereçam a memória de forma direta e por deslocamento, além de ser usado no *load* imediato. O bit R é setado para os endereçamentos indireto e por deslocamento.

Tabela 6 – Formato das instruções do tipo M.

Formato	***cond***	**op0 (10)**	**S**	**I**	**R**	**RN**	**RD/RM**	**End/Imediato**
Bits	31:28	27:26	25	24	23	22:18	17:13	12:0

Fonte: (PARK, 2025)

2.3.4 Instruções de Tipo A

As instruções alternativas, ou de tipo A, são aquelas que não se enquadram nas funcionalidades dos outros tipos de instruções descritos. Essas instruções estão listadas na Tabela 7.

Tabela 7 – Instruções de tipo A.

Mnemônico	**op1**	**Descrição**
NOP	0000	Não faz nada
HLT	0001	Inibe a atualização do PC
IN	0010	Guarda valor de entrada do FPGA em RD
OUT	0011	Faz o display do valor de RN no FPGA

Fonte: (PARK, 2025)

A Tabela 8 mostra o formato das instruções de tipo A. O *opcode* tem tamanho de 6 bits e é composto pelos campos op0 e op1. Cada instrução de tipo A tem um campo op1 único, conforme visto na Tabela 7. Os bits 21 e 17 contêm os registradores RD e RN utilizados nas instruções IN e OUT.

Tabela 8 – Formato das instruções do tipo A.

Formato	***cond***	**op0**	**op1**	**RN/RD**	**x**
Bits	31:28	27:26	25:22	21:17	16:0

Fonte: (PARK, 2025)

2.4 Organização da Memória

O processador está estruturado de acordo com a arquitetura de Harvard, portanto conta com memórias de instruções e dados separadas, conforme visto nas Figuras 1 e 2. Ambas as memórias armazenam palavras de 32 bits por endereço. Apesar de seus tamanhos poderem ser alterados, para fins desta implementação, a memória de instruções foi configurada com 2^8 endereços, e a memória de programa com 2^{10} endereços.

Como visto na seção 2.3.3, o conjunto de instruções do processador não conta com instruções de manuseio de pilha, por exemplo, o que exige um tratamento especial por parte do programador ou compilador. Igualmente, não há um *stack pointer* (SP), registrador específico para o tratamento de pilha na memória. Conforme já discutido, o único registrador reservado entre os de propósito geral é R31, o *link register*, que tem tratamento específico via *hardware*. Este sempre guarda o valor do *program counter* incrementado de uma unidade diante de uma instrução BL.

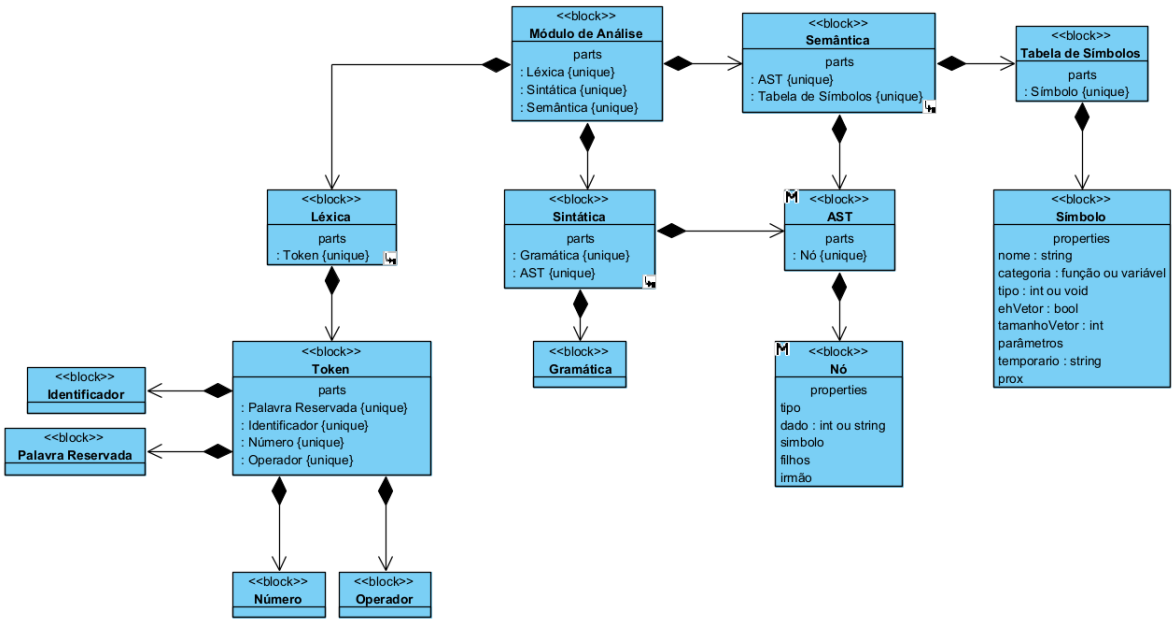
3 Compilador: Fase de Análise

O compilador implementado foi desenvolvido para a linguagem de programação `C-` (LOUDEN, 2004), um subconjunto da linguagem C. Nesta seção, são descritas as convenções léxicas, sintáticas e semânticas que a linguagem define, assim como suas implementações.

3.1 Modelagem

A fase de análise do compilador consiste na etapa em que a estrutura e o sentido do código-fonte são investigados. Essa análise é feita em três níveis: primeiro, é conferido se o texto do código-fonte está escrito de maneira correta, de acordo com as convenções da linguagem; então, é analisada a estrutura do código, de modo que o fluxo esteja de acordo com a gramática da linguagem; e, por último, é verificado o sentido do código de entrada, tanto no que diz respeito às funções e variáveis utilizadas, quanto quanto aos tipos de expressões e outras estruturas. Essas análises ganham o nome de léxica, sintática e semântica, respectivamente. A Figura 3 mostra o diagrama de blocos da fase de análise do compilador.

Figura 3 – Diagrama de blocos da fase de análise.

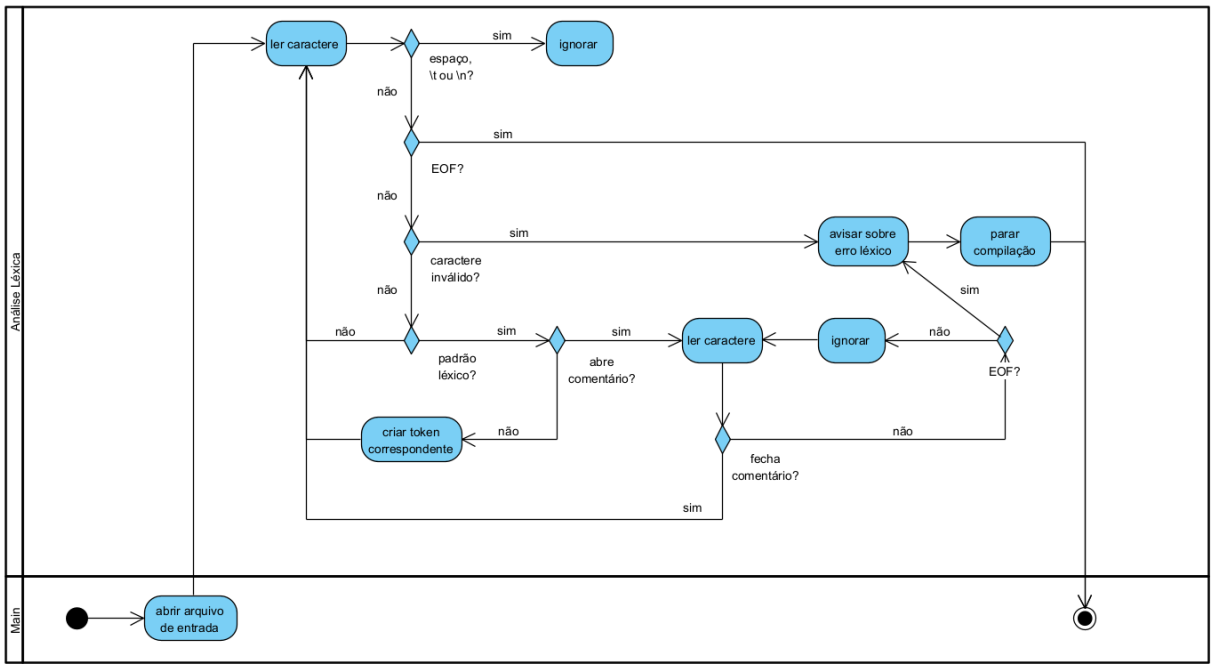


Fonte: A autora (2026).

A análise léxica, pilar inicial da fase de análise, lê o código-fonte e o divide em *tokens*, de acordo com as convenções léxicas da linguagem. Esses *tokens* desempenham

papel crucial na etapa de análise sintática, que analisa a ordem e estrutura dos tokens gerados. A Figura 4 mostra o seu diagrama de atividades, o qual é melhor comentado na seção 3.2.

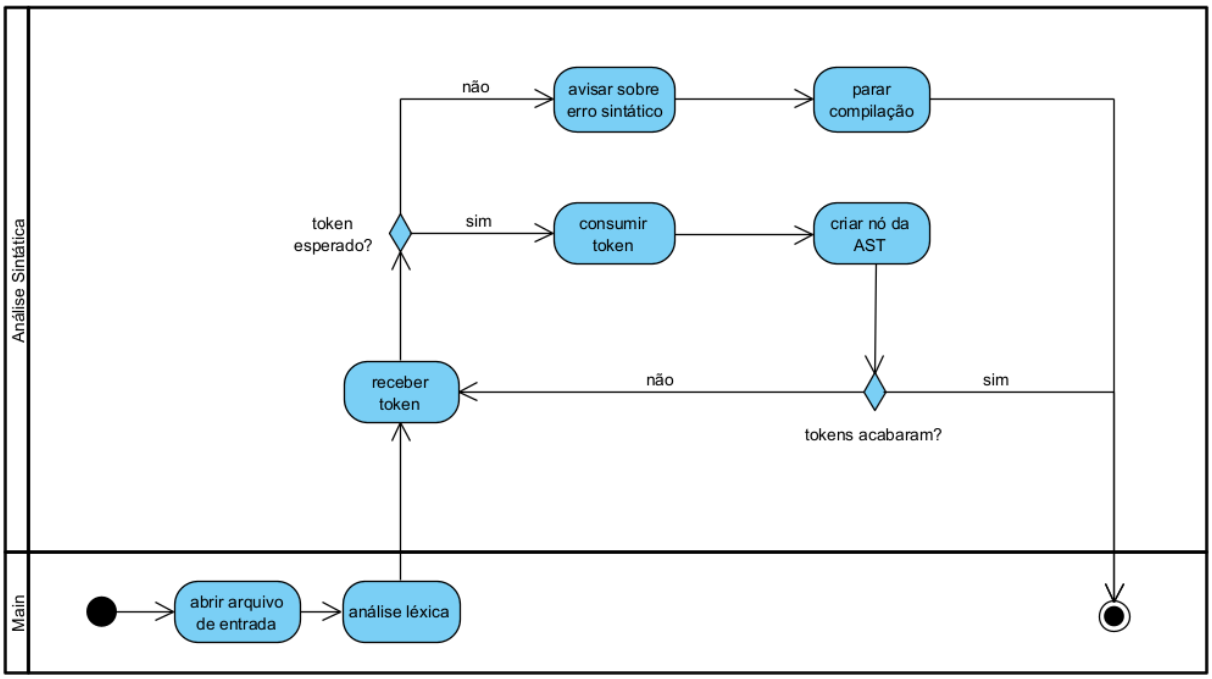
Figura 4 – Diagrama de atividades da análise léxica.



Fonte: A autora (2026).

A análise sintática é feita sobre os *tokens* gerados na etapa anterior da fase de análise. Ela tem o papel de verificar se a ordem na qual esses *tokens* aparecem está de acordo com a estrutura prevista, a qual é ditada pela gramática da linguagem. Além disso, é durante o processo de análise sintática que é construída a árvore de sintaxe abstrata (*abstract syntax tree*, ou AST), estrutura de dados utilizada para representar e classificar os elementos do código-fonte. A AST é muito importante na etapa de análise semântica, assim como na etapa de geração de código intermediário. A Figura 5 mostra o diagrama de atividades da etapa de análise sintática, o qual é melhor comentado na seção 3.3.

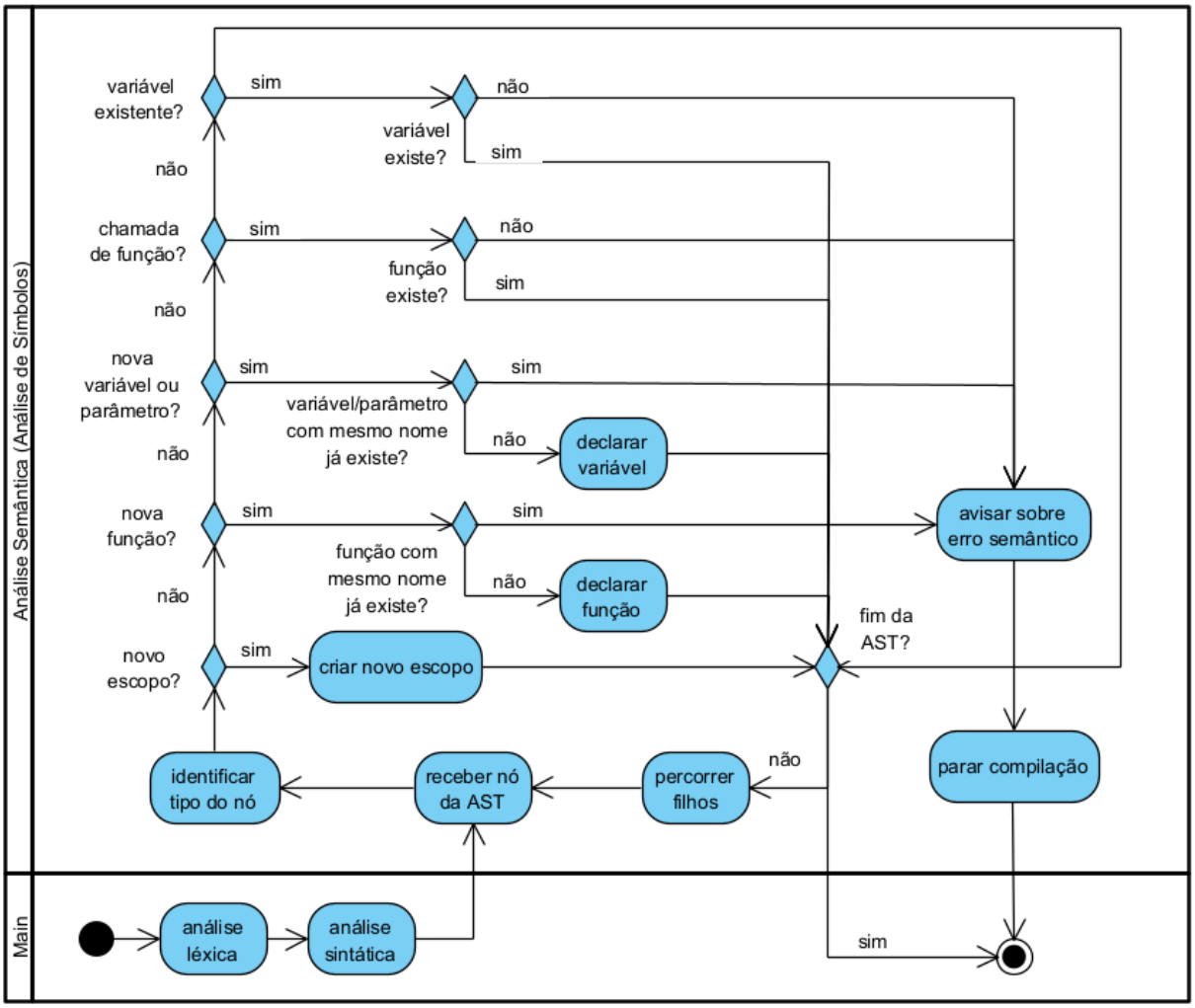
Figura 5 – Diagrama de atividades da análise sintática.



Fonte: A autora (2026).

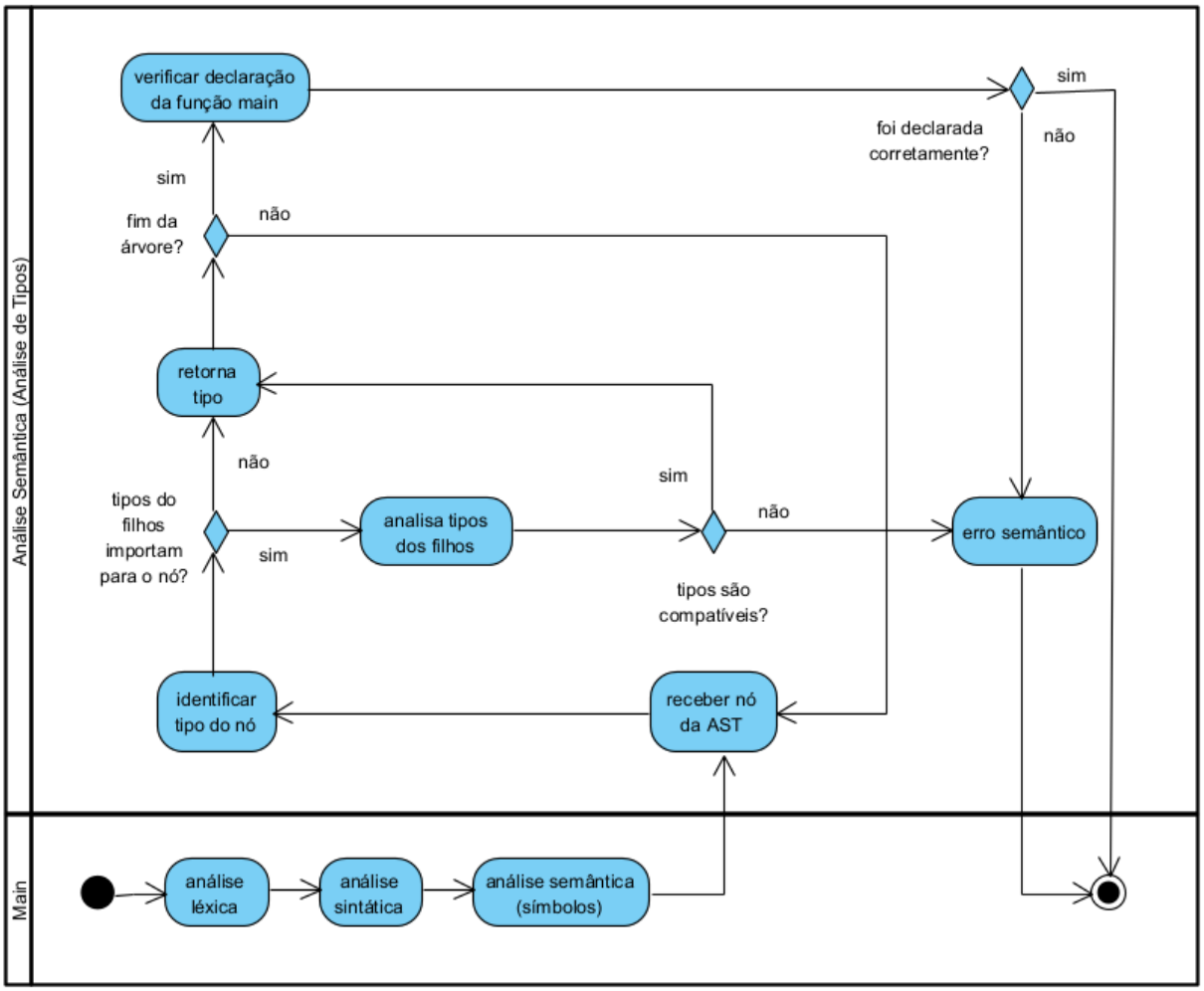
Já a análise semântica é o processo e que é invetigado o sentido do código escrito. Ela é separada em duas rotinas principais: verificação de símbolos e de tipos. Na primeira rotina, a AST é percorrida de maneira que cada símbolo encontrado é analisado individualmente. Símbolos são funções e variáveis. Durante essa etapa, é construída a tabela de símbolos, uma outra estrutura de dados que armazena as principais informações sobre cada símbolo utilizado no código. A segunda rotina, por outro lado, utiliza a AST e a tabela de símbolos para verificar os tipos dos mais variados operandos, sejam variáveis, funções ou expressões, e conferir se não há inconsistências ou incompatibilidades. As Figuras 6 e 7 mostram os diagramas de atividades das rotinas de verificação de símbolos e de tipos, respectivamente, e serão melhor comentadas na seção 3.4.

Figura 6 – Diagrama de atividades da verificação de símbolos da análise semântica.



Fonte: A autora (2026).

Figura 7 – Diagrama de atividades da verificação de tipos da análise semântica.



Fonte: A autora (2026).

3.2 Análise Léxica

Conforme já explicado, a análise léxica tem o papel de gerar *tokens* a partir do código-fonte, o que é feito de acordo com as convenções léxicas da linguagem. A Tabela 9 mostra essas convenções para o `C-`.

Tabela 9 – Convenções léxicas da linguagem `C-`.

Terminais	`if` `else` `while` `return` `void` `int`

Símbolos	`+` `-` `*`
	`/` `<` `<=`
	`>` `>=` `==`
	`!=` `=` `;`
	`,` `()` `[]`
	`{}` `/*` `*/`

Identificador	`letra = a-zA-Z` `ID = letra[letra]*`
Número	`dígito = 0-9` `NUM = dígito[dígito]*`
Espaço em branco	`[espaço]` `\t` `\n`
Comentário	Texto entre `/*` e `*/` Não aninhados

Fonte: (LOUDEN, 2004)

Os terminais da linguagem são palavras reservadas que desempenham papéis específicos na estrutura lógica do programa. As palavras `if`, `else`, `while` e `return` são terminais que declaram estruturas condicionais, de repetição e retorno dentro da linguagem, enquanto as palavras `int` e `void` definem os dois tipos existentes no `C-`. Os símbolos da linguagem dizem respeito às operações aritméticas e relacionais, além de darem suporte para a definição e estruturação do código. A convenção léxica que diz respeito aos nomes de variáveis e funções é o identificador. No `C-`, conforme visto na Tabela 9, estes são descritos apenas por sequências de letras, sejam minúsculas ou maiúsculas. Outra convenção é o número, composto apenas por sequências de dígitos, descrevendo apenas números inteiros positivos.

O *scanner*, ou analisador léxico, lê o código-fonte caractere a caractere, criando *tokens* a partir dos padrões léxicos reconhecidos. A Figura 4 mostra esse processo. A cada caractere lido, é tomada uma decisão para se prosseguir com a análise. Espaços em branco, tabulações e quebras de linha são ignorados, ao passo que caracteres não definidos nas convenções léxicas do `C-` resultam em erros léxicos. Ao identificar o símbolo `/*` (abre comentário), o texto seguinte é ignorado até que o símbolo `*/` (fecha comentário) é identificado. Essa etapa resulta em erro léxico caso a leitura do arquivo fonte acabe

nesse intervalo. No demais casos, o *scanner* reconhece padrões léxicos, de acordo com as convenções da Tabela 9, e cria os tokens de identificador, número, terminal (palavra reservada) ou operador correspondentes. Esse processo só termina com sucesso uma vez que o arquivo fonte é lido por completo.

3.3 Análise Sintática

Durante a análise sintática, é feita a verificação da estrutura do código-fonte de acordo com a gramática da linguagem. A Tabela 10 mostra essa gramática para o `C-`.

Tabela 10 – Gramática da linguagem `C-`.

programa →	listaDeclaracoes
listaDeclaracoes →	listaDeclaracoes declaracao | declaracao
declaracao →	declaracaoVariaveis | declaracaoFuncao
declaracaoVariaveis →	tipoEspecificador `ID ;` | tipoEspecificador `ID [ NUM ] ;`
tipoEspecificador →	`int` | `void`
declaracaoFuncao →	tipoEspecificador `ID` (parametros) escopo
parametros →	listaParametros | `void`
listaParametros →	listaParametros `,` parametro | parametro
parametro →	tipoEspecificador `ID` | tipoEspecificador `ID [ ]`
escopo →	{ declaracoesLocais listaEscopo }
declaracoesLocais →	declaracoesLocais declaracaoVariaveis | `null`
listaEscopo →	listaEscopo corpo | `null`
corpo →	declaracaoExpressao | escopo | declaracaoSelecao | declaracaoIteracao | declaracaoRetorno
declaracaoExpressao →	expressao `;` | `;`
declaracaoSelecao →	`if` (expressao) corpo | `if` (expressao) corpo `else` corpo
declaracaoIteracao →	`while` (expressao) corpo
declaracaoRetorno →	`return ;` | `return` expressao `;`
expressao →	variavel `=` expressao | expressaoSimples
variavel →	`ID` | `ID` [expressao]
expressaoSimples →	expressaoSoma relacional expressaoSoma | expressaoSoma
relacional →	`<=` | `<` | `>` | `>=` | `==` | `!=`
expressaoSoma →	expressaoSoma soma termo | termo
soma →	`+` | `-`
termo →	termo mult fator | fator
mult →	`*` | `/`
fator →	(expressao)
chamadaFuncao →	`ID` (argumentos)
argumentos →	listaArgumentos
listaArgumentos →	listaArgumentos `,` expressao

Fonte: (LOUDEN, 2004)

O *parser*, ou analisador sintático, recebe os *tokens* gerados pelo *scanner* e verifica sua estrutura. Conforme visto no diagrama da Figura 5, ao receber o *token*, ele decide se é esperado ou não de acordo com a gramática da Tabela 10. Caso positivo, o *token* é consumido, ou seja, retirado da pilha de *tokens*, e é criado um nó na AST para a representação do elemento. Caso contrário, é indicado um erro sintático e a compilação é interrompida. Esse processo finaliza com sucesso uma vez que a pilha de *tokens* está vazia. Nesse ponto, a AST está completa e é possível prosseguir para a análise semântica.

3.4 Análise Semântica

A linguagem `C-` possui várias convenções semânticas (LOUDEN, 2004), listadas a seguir.

- Todas as variáveis e funções são declaradas antes do uso;
- Todo programa acaba com a declaração de uma função da forma `void main(void)`;
- Variáveis são inteiros simples ou vetores, não podem ser de tipo `void`;
- Funções podem ser de tipo `int` ou `void`;
- Funções de tipo `int` retornam, obrigatoriamente, um valor inteiro;
- Funções de tipo `void` não retornam valores;
- Funções podem ou não ter parâmetros;
- Parâmetros de funções são variáveis inteiras simples ou vetores;
- Parâmetros de tipo vetor são passados por referencia, ou seja, como ponteiros;
- Parâmetros de tipo vetor devem casar com uma variável tipo vetor durante a chamada da função;
- A função `int input(void)` tem nome reservado e realiza uma operação de entrada de valor;
- A função `void output(int x)` tem nome reservado e realiza a operação de saída, ou *display*, de valor.

A análise semântica tem o papel de verificar se o programa sendo compilado se encaixa nessas convenções. Para isso, ela é dividida na rotina de verificação de símbolos (que englobam funções e variáveis) e de tipos.

A primeira rotina da análise semântica, a verificação de símbolos, começa percorrendo a AST, conforme mostra o diagrama da Figura 6. Nesta etapa da análise, os únicos nós relevantes são os que contém ou referenciam símbolos ou geram novos escopos, a fim da construção da tabela de símbolos. Com isso, é primeiramente identificado o tipo do nó.

Caso o nó gere um novo escopo, ele é criado e todos os símbolos declarados pelos filhos são registrados nele. Se o nó declarar uma variável ou função, é verificado na tabela de símbolos se uma variável ou função de mesmo nome já existe. Caso positivo, é indicado erro semântico e a compilação é interrompida. Caso contrário, a variável ou função é declarada na tabela de símbolos, no escopo vigente.

Se o nó fizer referência a um símbolo, no caso do uso de variáveis ou chamadas de função, é verificado na tabela de símbolos se uma variável ou função de mesmo nome já existe. Caso negativo, é indicado erro semântico e a compilação é interrompida. Caso contrário, a análise continua. Esse processo é repetido até que todos os nós da AST tenham sido visitados. É interessante notar, também, que a AST é percorrida em pré-ordem, sendo que o nó raiz é o primeiro visitado, depois os filhos da esquerda para a direita e, por fim, os irmãos. Isso permite que nós que referenciam símbolos só sejam visitados após os nós que declaram esses símbolos, de maneira a evitar inconsistências na análise ou indicação de erros semânticos falsos.

Após esse processo, inicia-se a segunda rotina da análise semântica, a verificação de tipos. Nela, conforme mostra a Figura 7, a AST também é percorrida. Primeiro é identificado o tipo do nó, e então é decidido se o tipo importa. Caso positivo, é verificado se os tipos dos filhos são compatíveis. Um exemplo seria na verificação de um nó de atribuição, onde a variável do lado esquerdo e a expressão do lado direito, seja um número, chamada de função ou operação aritmética ou relacional, têm que ser de tipo inteiro. Se os tipos forem incompatíveis, ou no caso do nó de atribuição, de tipo `void`, é indicado erro semântico e a compilação é interrompida. Caso contrário, o tipo é retornado. Caso o o tipo do nó não importe, é retornado o tipo `void`. Esse processo é repetido até que todos os nós da árvore sejam percorridos. Diferente da primeira rotina da análise semântica, em que a AST é percorrida em pré-ordem, nesta etapa, a AST é percorrida em pós-ordem. Dessa maneira, é primeiramente feita a análise dos filhos e o nó raiz simplemente propaga esse tipo.

Após a verificação dos tipos, é analisada a situação da função `main`. Isso é feito verificando-se o primeiro símbolo do primeiro escopo (global) declarado na tabela de símbolos, já que sua delaração como última função do programa determina sua posição. Vale destacar que essa análise é feita tanto para o nome da função, mas também para tipo e quantidade de parâmetros, de forma que ela seja, de fato, uma função do formato `void main(void)`.

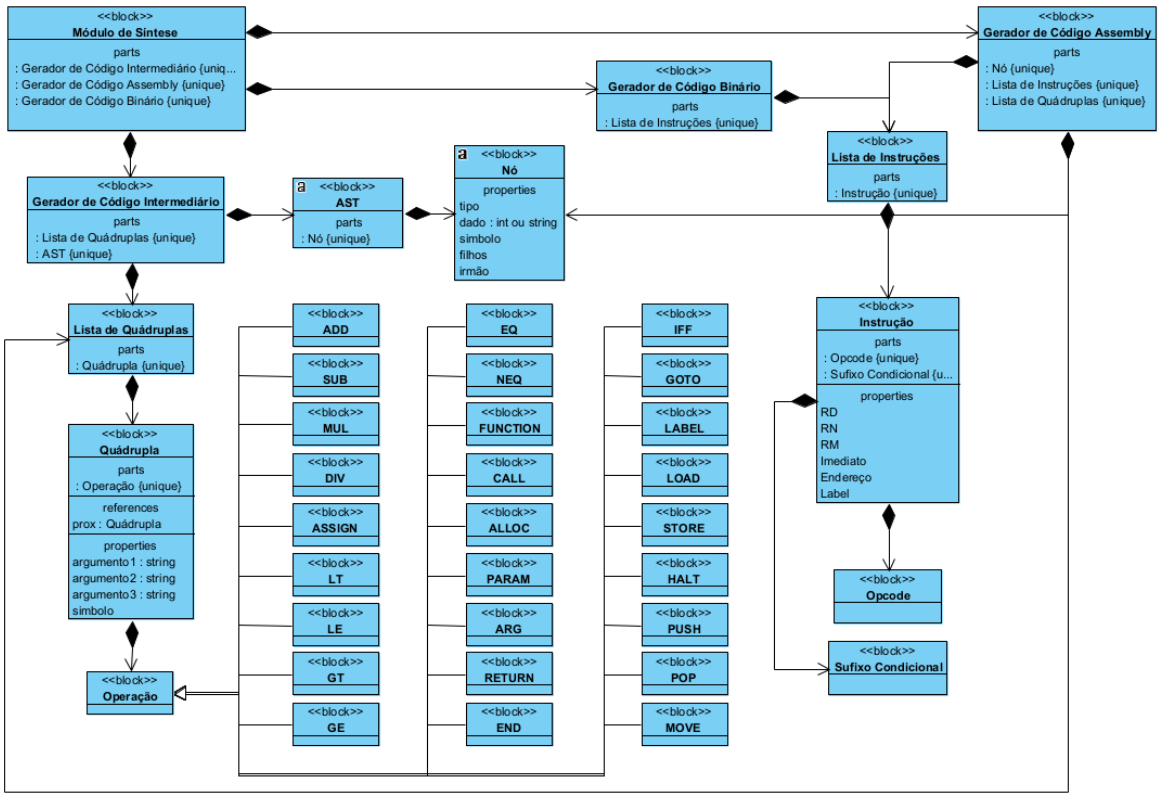
4 Compilador: Fase de Síntese

Dada a análise feita na primeira fase do compilador, a fase de síntese, detalhada nesta seção, tem como objetivo a geração de código, de fato. Ao final, o propósito do projeto é a geração de linguagem de máquina de acordo com o conjunto de instruções do processador descrito na seção 2.

4.1 Modelagem

A fase de síntese do compilador consiste na etapa da transformação do código-fonte em código executável, de fato. Ela consiste em três etapas: a geração de código intermediário, um tipo de código mais abstrato considerado muitas vezes a linearização da AST; a geração de código assembly, a qual já diz respeito ao conjunto de instruções do processador utilizado; e, por último, a geração do código executável, em binário. Conforme visto no diagrama de blocos da Figura 8, a primeira etapa percorre a AST para gerar uma lista de quádruplas, que são uma maneira de representação do código intermediário. Durante a geração de código assembly a lista de quádruplas é percorrida de maneira que cada tipo de quádrupla seja traduzida para uma ou mais linhas de código assembly. Por fim, para a geração de código binário, a lista de instruções assembly gerada na etapa anterior é percorrida, e cada instrução é individualmente traduzida para uma sequência binária de 32 bits.

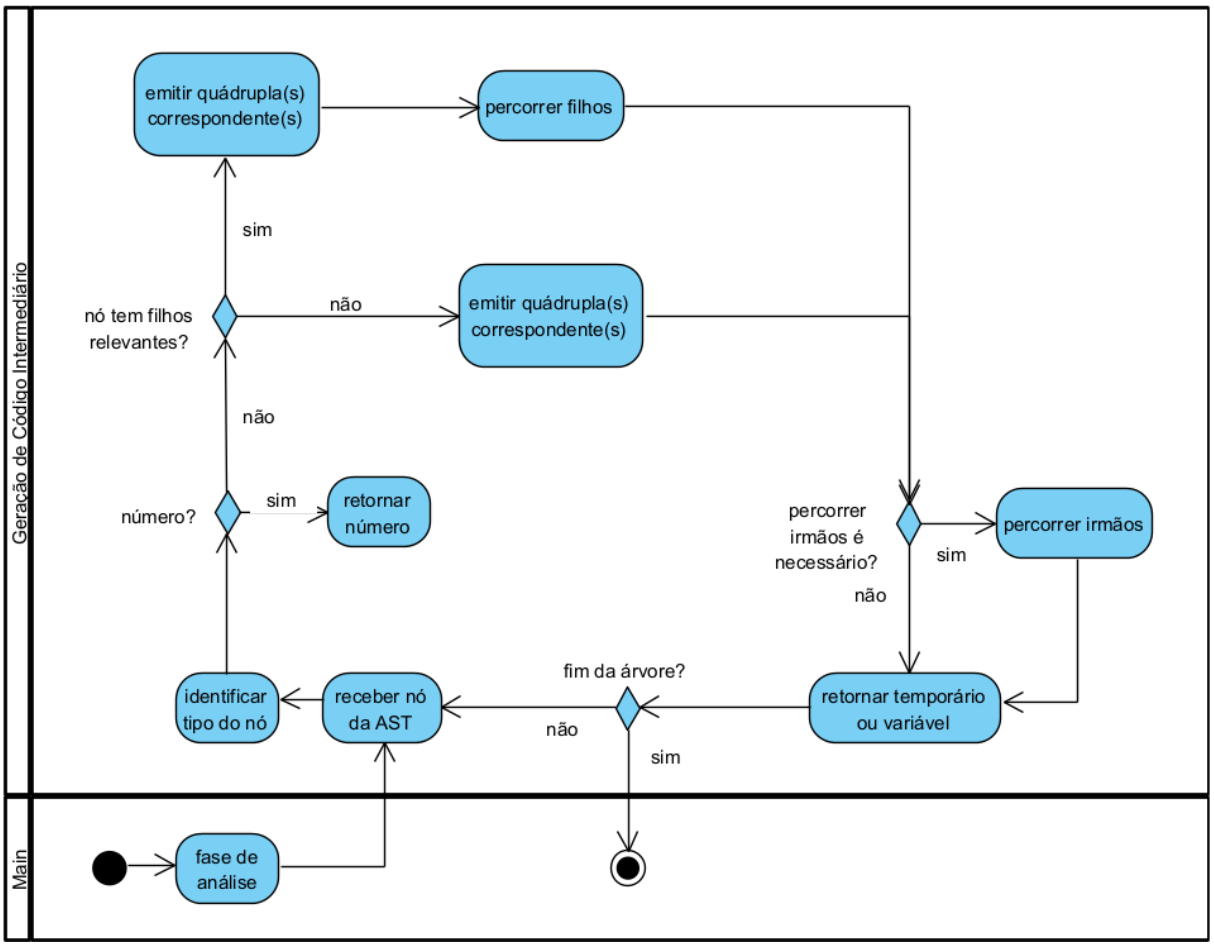
Figura 8 – Diagrama de blocos da fase de síntese.



Fonte: A autora (2026).

O código intermediário é uma maneira geral de se representar o programa, de forma que possa ser compatível com várias arquiteturas diferentes. A maneira mais comum de representação do código intermediário é o TAC (*three address code*, ou código de três endereços), no qual cada linha de código tem três endereços, no máximo. Para este projeto, foi usado o formato de quádruplas, um tipo comum de TAC. Essa etapa envolve a varredura da AST e geração de quádruplas correspondentes a cada nó da árvore. Essas quádruplas são agrupadas em uma lista sequencial, de maneira que a geração de código intermediário é muitas vezes observada como a linearização da AST. A Figura 9 mostra o diagrama de atividades da geração de código intermediário, o qual é melhor detalhado na seção 4.2.

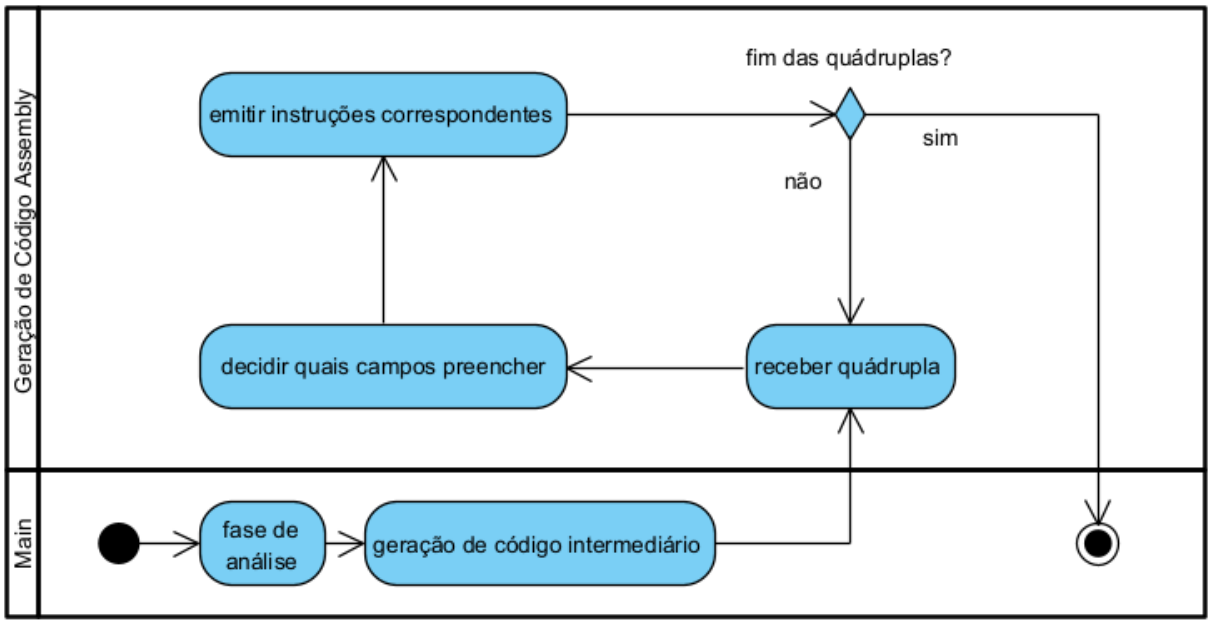
Figura 9 – Diagrama de atividades da geração de código intermediário.



Fonte: A autora (2026).

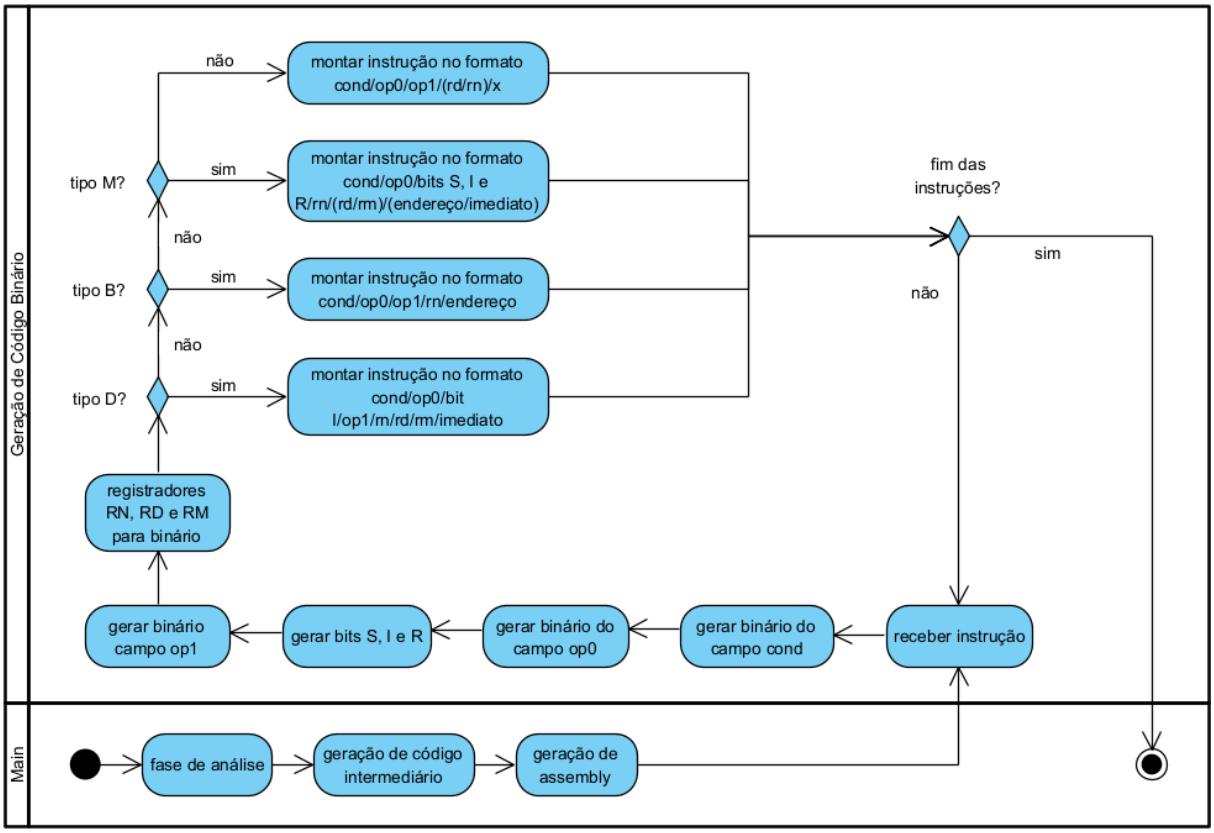
Cada quádrupla é convertida diretamente a uma ou mais linhas de código assembly, a depender do seu tipo. O código assembly gerado pelo compilador se baseia no conjunto de instruções descrito na seção 2.3. A conversão de assembly para binário é direta, e também segue o padrão já definido. As Figuras 10 e 11 mostram os diagramas de atividades das gerações de código assembly e binário, respectivamente, e são melhor comentadas nas seções 4.3 e 4.4.

Figura 10 – Diagrama de atividades da geração de código assembly.



Fonte: A autora (2026).

Figura 11 – Diagrama de atividades da geração de código binário.



Fonte: A autora (2026).

4.2 Geração de Código Intermediário

O código intermediário gerado pelo compilador, conforme já discutido, é representado no formato de quádruplas. Para isso, foram definidos tipos fixos de quádruplas, conforme mostra a Tabela 11. A cada tipo foi atribuída uma cor diferente, a fim de deixar visível como uma quádrupla se traduz em linhas de código assembly e, posteriormente, binário na comparação de códigos apresentada na seção 5.

Tabela 11 – Formatos das quádruplas.

Quádruplas				
Operação	**Arg 1**	**Arg 2**	**Arg 3**	**Descrição**
ADD	operando	operando	destino	soma
SUB	operando	operando	destino	subtração
MUL	operando	operando	destino	multiplicação
DIV	operando	operando	destino	divisão
ASSIGN	operando	-	destino	atribuição
LT	operando	operando	destino	menor
LE	operando	operando	destino	menor ou igual
GT	operando	operando	destino	maior
GE	operando	operando	destino	maior ou igual
EQ	operando	operando	destino	igual
NEQ	operando	operando	destino	diferente
FUNCTION	ID da função	tipo	-	começo de função
CALL	ID da função	n° argumentos	destino	chamada de função
ALLOC	ID da função	ID da variável	-	alocação de memória
PARAM	ID da função	ID da variável	-	parâmetro de função
ARG	origem	-	-	argumento de função
RETURN	origem	-	-	retorno de função
END	ID da função	-	-	fim de função
IFF	origem	-	label	salto condicional
GOTO	-	-	label	salto incondicional
LABEL	-	-	label	atribuição de label
LOAD	ID da variável	índice acessado	destino	carrega variável
STORE	ID da variável	índice acessado	origem	guarda variável
HALT	-	-	-	para processamento
PUSH	-	-	origem	empilha argumento
POP	-	-	destino	desempilha argumento
MOVE	origem	-	destino	move argumento

Fonte: A autora (2026).

A geração de código intermediário, conforme mostra o diagrama da Figura 9, começa com a varredura da AST. Caso o nó represente um número, ele simplemente retorna esse número. Caso contrário, é analisado se o nó tem filhos relevantes. Esse é o caso de nós como declaração de função, operações aritméticas, relacionais ou de atribuição, e declarações condicionais, de repetição ou de retorno. Seja esse o caso, as quádruplas correspondentes são emitidas, e depois os filhos são percorridos. Caso contrário, as quádruplas são emitidas, mas os filhos não são percorridos. Ao final, também é analisado se os irmãos do nó devem ser percorridos, o que é feito por meio de um argumento passado para a função de varredura. Então, um temporário gerado durante a emissão de quádruplas ou variável que o nó representa são retornados.

4.3 Geração de Código Assembly

A geração de código assembly é feita com base no conjunto de instruções descrito na seção 2.3. Na Tabela 12, define-se o modo como cada instrução desse código é escrito.

Tabela 12 – Formatos das instruções assembly.

Instruções Assembly	
Operação	**Formato**
ADD	`ADD RD, RN, RM/#Imediato`
ADDS	`ADDS RD, RN, RM/#Imediato`
SUB	`SUB RD, RN, RM/#Imediato`
SUBS	`SUBS RD, RN, RM/#Imediato`
RSB	`RSB RD, RN, RM/#Imediato`
MUL	`MUL RD, RN, RM/#Imediato`
UDIV	`UDIV RD, RN, RM/#Imediato`
NOT	`NOT RD, RN, RM/#Imediato`
AND	`AND RD, RN, RM/#Imediato`
ORR	`ORR RD, RN, RM/#Imediato`
EOR	`EOR RD, RN, RM/#Imediato`
MOV	`MOV RD, RN, RM/#Imediato`
CMP	`CMP RD, RN, RM/#Imediato`
B	`B label`
BL	`BL label`
BX	`BX RN`
LDR (Imediato)	`LDR RD, #Imediato`
LDR (Direto)	`LDR RD, =Endereço`
LDR (Indireto)	`LDR RD, RN`

continua na próxima página

continuação da Tabela 12

Instruções Assembly	
Operação	**Formato**
LDR (Deslocamento)	`LDR RD, [RN, #Imediato]`
STR (Direto)	`STR RM, =Endereço`
STR (Indireto)	`STR RM, RN`
STR (Deslocamento)	`STR RM, [RN, #Imediato]`
NOP	NOP
HLT	HLT
IN	`IN RD`
OUT	`OUT RN`

Fonte: A autora (2026).

A geração de código assembly, conforme mostra a Figura 10, percorre a lista de quádruplas. Dada uma determinada quádrupla, o compilador decide quais campos da instrução preencher. Por exemplo, no caso de uma quádrupla `(ADD, R1, 2, R3)`, seria gerada uma instrução ADD com valor intermediário onde os campos RD, RN e imediato seriam preenchidos. Por outro lado, se a quádrupla fosse `(ADD, R1, R2, R3)`, os campos RD, RN e RM da instrução ADD seriam preenchidos.

Uma quádrupla não necessariamente corresponde diretamente a uma única instrução assembly. Existem tipos de quádruplas que são traduzidas em mais de uma instrução. Um exemplo disso são as quádruplas do tipo PUSH, que simulam o empilhamento e desempilhamento de valores na memória. Conforme explicitado na seção 2.4, não existem instruções específicas no conjunto de instruções do processador que façam a manipulação de pilha. Para haver correspondência com a quádrupla gerada, portanto, são emitidas duas instruções: uma que guarda o valor a ser empilhado no endereço do *stack pointer*, e outra que incrementa o valor desse registrador. A mesma lógica pode ser observada em quádruplas do tipo POP ou ARG, por exemplo.

4.4 Geração de Código Executável

O código binário é uma tradução direta do código assembly. Portanto, nenhuma lógica adicional de geração de novas linhas de código é implementada.

Conforme mostra o diagrama da Figura 11, o gerador de código binário, ou montador, faz a tradução do assembly ao percorrer a lista de instruções gerada na etapa anterior da síntese. É feito, primeiramente, o cálculo dos campos cond, op0, op1, dos bits S, I e R e dos registradores RD, RN e RM para todas as instruções. Mesmo que algum desses campos esteja vazio, esse cálculo não impõe que o campo será usado na montagem da

instrução em binário. Em seguida, pelo mnemônico da instrução, é identificado o seu tipo, e então feita a montagem de acordo com o que é relevante. Por exemplo, instruções do tipo M não fazem o uso do campo op1, e instruções que não sejam de tipo M não fazem o uso dos bits S e R. Esse processo é repetido até que a lista de instruções tenha sido percorrida por inteiro.

4.5 Gerenciamento de Memória

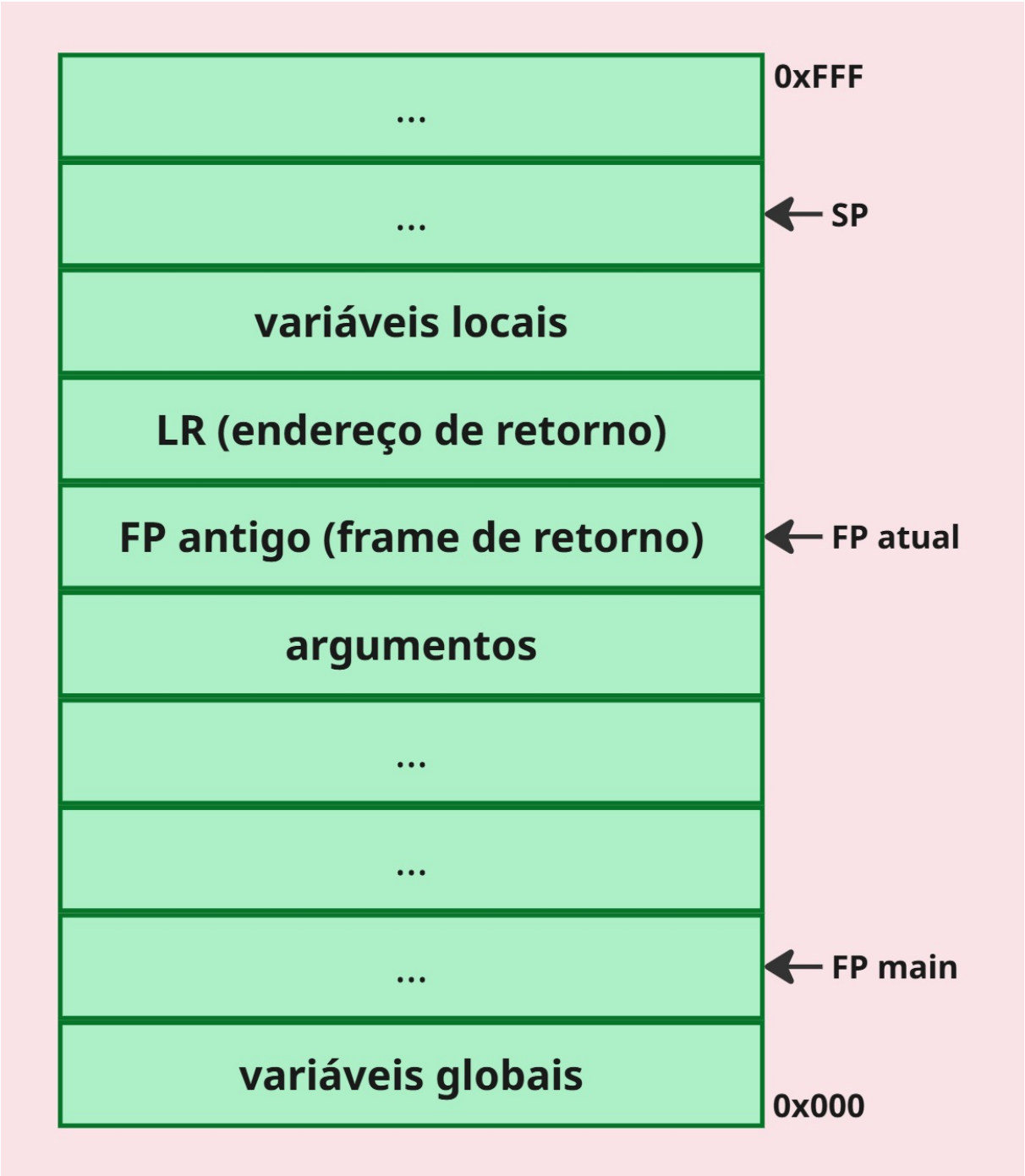
Para o tratamento de chamadas a funções, variáveis globais e locais, foi criada uma convenção para o gerenciamento de memória, conforme mostra a Figura 12. Foi definido que cada instância de execução de uma função do programa tem um espaço de memória para escrita definido por uma base fixa, cujo endereço é guardado pelo registrador especial *frame pointer* (FP), R29, e pelo endereço contido no *stack pointer* (SP), R30, que é móvel. A memória foi modelada de forma que funcione como uma pilha, com *frames* de funções podendo ser empilhados conforme necessidade.

O programa começa alocando espaço na memória para todas as variáveis globais declaradas. Isso é feito incrementando-se o valor de SP, que começa em zero. Ao fim dessa alocação, o valor de SP é considerado o FP da função main, a partir de onde a pilha da memória pode começar a receber variáveis locais da função main e seus diferentes *frames*, que correspondem a chamadas de funções.

Com a exceção das funções input e output, que têm instruções específicas destinadas ao seu tratamento, chamadas de funções seguem o mesmo padrão. Em primeiro lugar, empilha-se na memória o conteúdo dos registradores em uso, já que podem ser sobrescritos no processamento da função chamada. Em seguida, são passados empilhados os argumentos da função chamada e efetuado o desvio para a função, salvando-se o valor de retorno no *link register*. No começo de toda nova função, o valor do FP anterior é guardado no endereço atual de SP e então atualizado para esse mesmo endereço de SP. Essa é considerada a base do *frame* atual. Então, é empilhado também o valor de LR, para o caso de mais uma chamada de função ocorrer e sobrescrever o seu conteúdo. A partir de FP + 2 são, enfim, alocadas as variáveis locais e efetuadas outras chamadas de funções, se for o caso.

Para a retomada de contexto, é carregado o valor de FP + 1 em LR, o valor de FP + 0 em FP e efetua-se o desvio para o valor de LR. Na volta da função, é subtraído do conteúdo de SP a quantidade de argumentos que foram empilhados anteriormente à chamada da função. No caso de retorno de valores, para funções do tipo inteiro, foi reservado o registrador R0 como *return register*, ou seja, como registrador de retorno.

Figura 12 – Modelo de gerenciamento de memória.



Fonte: A autora (2026).

5 Exemplos

5.1 Fatorial Recursivo

A Figura 13 mostra uma implementação de fatorial recursivo em C-. A Tabela 13 mostra os códigos intermediário, assembly e binário gerados pelo compilador, assim como a correspondência entre os três. Esse exemplo deixa claro o mecanismo de chamada de funções, assim como a preservação e restauração dos valores dos registradores na troca de contexto de funções.

Figura 13 – Programa para cálculo de fatorial em C-.

```
1     int fact(int n) {
2         if (n <= 0) return 1;
3         return n * fact(n - 1);
4     }
5
6     void main(void) {
7         int n;
8         n = input();
9         output(fact(n));
10    }
11
```

Fonte: (LOUDEN, 2004).

Tabela 13 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de cálculo de fatorial.

Quádrupla	Assembly	Binário
	NOP	11101100000000000000000000000000
	MOV R0, #0	11100010101100000000000000000000
	MOV R29, #0	11100010101100000111010000000000
	MOV R30, #0	11100010101100000111100000000000
	MOV R31, #0	11100010101100000111110000000000
	B main	11100100000000000000000000101001
	fact:	
	STR R29, R30	11101010111110111010000000000000
(FUNCTION,	MOV R29, R30	11100000101100000111011111000000
fact, int, -)	ADD R30, R30, #1	11100010000011110111100000000001
	STR R31, R30	11101010111110111110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001

continua na próxima página

continuação da Tabela 13

Quádrupla	Assembly	Binário
(PARAM, fact, n, int)		
(LOAD, n, -, t1)	SUB R2, R29, #1 LDR R1, R2	11100010001011101000100000000001 11101000100010000010000000000000
(LE, t1, 0, t2)	CMP R1, #0	11100010110000001000000000000000
(IFF, t2, -, L0)	BGT L0	11000100000000000000000000010100
(RETURN, 1, -, -)	MOV R0, #1 LDR R31, [R29, #1] MOV R30, R29 LDR R29, R30 BX R31	11100010101100000000000000000001 11101001111101111110000000000001 11100000101100000111101110100000 11101000111110111010000000000000 11100101111110000000000000000000
(LABEL, -, -, L0)	L0: NOP	 11101100000000000000000000000000
(PUSH, -, -, t1)	STR R1, R30 ADD R30, R30, #1	11101010111110000010000000000000 11100010000011110111100000000001
(SUB, t1, 1, t2)	SUB R2, R1, #1	11100010001000001000100000000001
(ARG, t2, -, -)	STR R2, R30 ADD R30, R30, #1	11101010111110000100000000000000 11100010000011110111100000000001
(CALL, fact, 1, t0)	BL fact SUB R30, R30, #1	11100110000000000000000000000110 11100010001011110111100000000001
(POP, -, -, t1)	SUB R30, R30, #1 LDR R1, R30	11100010001011110111100000000001 11101000111110000010000000000000
(MOVE, t0, -, t2)	MOV R2, R0	11100000101100000000100000000000
(MUL, t1, t2, t2)	MUL R2, R1, R2	11100000010100001000100001000000
(RETURN, t2, -, -)	MOV R0, R2 LDR R31, [R29, #1] MOV R30, R29 LDR R29, R30 BX R31	11100000101100000000000001000000 11101001111101111110000000000001 11100000101100000111101110100000 11101000111110111010000000000000 11100101111110000000000000000000
	LDR R31, [R29, #1] MOV R30, R29	11101001111101111110000000000001 11100000101100000111101110100000

continua na próxima página

continuação da Tabela 13

Quádrupla	Assembly	Binário
(END, fact, -, -)	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111111000000000000000000
(FUNCTION, main, void, -)	main: MOV R29, R30	11100000101100000111011111000000
(ALLOC, main, n, -)	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, input, 0, t0)	IN R0	11101100100000000000000000000000
	NOP	11101100000000000000000000000000
(MOVE, t0, -, t1)	MOV R1, R0	11100000101100000000010000000000
(STORE, n, -, t1)	STR R1, [R29, #0]	11101011111101000010000000000000
(PUSH, -, -, t1)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(ARG, t1, -, -)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, fact, 1, t0)	BL fact	11100110000000000000000000000110
	SUB R30, R30, #1	11100010001011110111100000000001
(POP, -, -, t1)	SUB R30, R30, #1	11100010001011110111100000000001
	LDR R1, R30	11101000111110000010000000000000
(MOVE, t0, -, t2)	MOV R2, R0	11100000101100000000100000000000
(MOVE, t2, -, t0)	MOV R0, R2	11100000101100000000000001000000
(CALL, output, 0, -)	OUT R0	11101100110000000000000000000000
	NOP	11101100000000000000000000000000
(END, main, -, -)		
(HALT, -, -, -)	HLT	11101100010000000000000000000000

Fonte: A autora (2026).

5.2 Maior Divisor Comum

A Figura 14 mostra uma implementação de cálculo de maior divisor comum em C- pelo algoritmo de Euclides. A Tabela 14 mostra os códigos intermediário, assembly e binário gerados pelo compilador, assim como a correspondência entre os três. Esse exemplo deixa claro o mecanismo de chamada de funções, assim como a recursão.

Figura 14 – Programa para cálculo de maior divisor comum em C-.

```
1     int gcd(int u, int v) {
2         if (v == 0) return u;
3         else return gcd(v, u-u/v*v);     /* u-u/v*v = u % v */
4     }
5
6     void main(void) {
7         int x;
8         int y;
9         x = input();
10        y = input();
11        output(gcd(x, y));
12    }
13
```

Fonte: (LOUDEN, 2004).

Tabela 14 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de cálculo de maior divisor comum.

Quádrupla	Assembly	Binário
	NOP	11101100000000000000000000000000
	MOV R0, #0	11100010101100000000000000000000
	MOV R29, #0	11100010101100000111010000000000
	MOV R30, #0	11100010101100000111100000000000
	MOV R31, #0	11100010101100000111110000000000
	B main	11100100000000000000000000101001
(FUNCTION, gcd, int, -)	gcd:	
	STR R29, R30	11101010111110111010000000000000
	MOV R29, R30	11100000101100000111011111000000
	ADD R30, R30, #1	11100010000011110111100000000001
	STR R31, R30	11101010111110111110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(PARAM, gcd, u, int)		
(PARAM, gcd, v, int)		

continua na próxima página

continuação da Tabela 14

Quádrupla	Assembly	Binário
(LOAD, v, -, t1)	SUB R2, R29, #1	11100010001011101000100000000001
	LDR R1, R2	11101000100010000010000000000000
(EQ, t1, 0, t2)	CMP R1, #0	11100010110000001000000000000000
(IFF, t2, -, L0)	BNE L0	00010100000000000000000000010111
(LOAD, u, -, t2)	SUB R3, R29, #2	11100010001011101000110000000010
	LDR R2, R3	11101000100011000100000000000000
(RETURN, t2, -, -)	MOV R0, R2	11100000101100000000000001000000
	LDR R31, [R29, #1]	11101001111101111110000000000001
	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111110000000000000000000
(GOTO, -, -, L1)	B L1	11100000101100000000000001000000
(LABEL, -, -, L0)	L0:	
	NOP	11101100000000000000000000000000
(PUSH, -, -, t1)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(ARG, t1, -, -)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(LOAD, u, -, t2)	SUB R3, R29, #2	11100010001011101000110000000010
	LDR R2, R3	11101000100011000100000000000000
(DIV, t2, t1, t3)	UDIV R3, R2, R1	11100000011000010000110000100000
(MUL, t3, t1, t3)	MUL R3, R3, R1	11100000010100011000110000100000
(SUB, t2, t3, t3)	SUB R3, R2, R3	11100000001000010000110001100000
(ARG, t3, -, -)	STR R3, R30	11101010111110000110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, gcd, 2, t0)	BL gcd	11100110000000000000000000000110
	SUB R30, R30, #2	11100010001011110111100000000010
(POP, -, -, t1)	SUB R30, R30, #1	11100010001011110111100000000001
	LDR R1, R30	11101000111110000010000000000000

continua na próxima página

continuação da Tabela 14

Quádrupla	Assembly	Binário
(MOVE, t0, -,	MOV R2, R0	11100000101100000000010000000000
(RETURN, t2, -, -)	MOV R0, R2	11100000101100000000000001000000
	LDR R31, [R29, #1]	11101001111101111110000000000001
	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111111000000000000000000
(LABEL, -, -, L1)	L1:	
	NOP	11101100000000000000000000000000
(END, gcd, -, -)	LDR R31, [R29, #1]	11101001111101111110000000000001
	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111111000000000000000000
(FUNCTION, main, void, -)	main:	
	MOV R29, R30	11100000101100000111011111000000
(ALLOC, main, x, -)	ADD R30, R30, #1	11100010000011110111100000000001
(ALLOC, main, y, -)	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, input, 0, t0)	IN R0	11101100100000000000000000000000
	NOP	11101100000000000000000000000000
(MOVE, t0, -, t1)	MOV R1, R0	11100000101100000000010000000000
(STORE, x, -, t1)	STR R1, [R29, #0]	11100000101100000000010000000000
(CALL, input, 0, t0)	IN R0	11101100100000000000000000000000
	NOP	11101100000000000000000000000000
(MOVE, t0, -, t2)	MOV R2, R0	11100000101100000000010000000000
(STORE, y, -, t2)	STR R2, [R29, #1]	11101011111101000100000000000001
(PUSH, -, -, t1)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(PUSH, -, -, t2)	STR R2, R30	11101010111110000100000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
	STR R1, R30	11101010111110000010000000000000

continua na próxima página

continuação da Tabela 14

Quádrupla	Assembly	Binário
(ARG, t1, -,	ADD R30, R30, #1	11100010000011110111100000000001
(ARG, t2, -,	STR R2, R30	11101010111110000100000000000000
-)	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, gcd, 2,	BL gcd	11100110000000000000000000000110
t0)	SUB R30, R30, #2	11100010001011110111100000000010
(POP, -, -,	SUB R30, R30, #1	11100010001011110111100000000001
t2)	LDR R2, R30	11101000111110000100000000000000
(POP, -, -,	SUB R30, R30, #1	11100010001011110111100000000001
t1)	LDR R1, R30	11101000111110000100000000000000
(MOVE, t0, -,	MOV R3, R0	11100000101100000000110000000000
t3)		
(MOVE, t3, -,	MOV R0, R3	11100000101100000000000001100000
t0)		
(CALL, output,	OUT R0	11101100110000000000000000000000
0, -)	NOP	11101100000000000000000000000000
(END, main, -,		
-)		
(HALT, -, -,	HLT	11101100010000000000000000000000
-)		

Fonte: A autora (2026).

5.3 *Sort*

A Figura 15 mostra uma implementação de fatorial recursivo em C-. A Tabela 15 mostra os códigos intermediário, assembly e binário gerados pelo compilador, assim como a correspondência entre os três. Esse exemplo deixa claro o mecanismo de chamada de funções, assim como a passagem de vetores como parâmetros de funções.

Figura 15 – Programa para ordenação de vetor em C-.

```
1   int x[10];
2
3   int minloc(int a[], int low, int high) {
4       int i; int x; int k;
5       k = low;
6       x = a[low];
7       i = low + 1;
8       while(i < high) {
9           if(a[i] < x) {
10              x = a[i];
11              k = i;
12          }
13          i = i + 1;
14      }
15      return k;
16  }
17
18  void sort(int a[], int low, int high) {
19      int i; int k;
20      i = low;
21      while(i < high-1) {
22          int t;
23          k = minloc(a, i, high);
24          t = a[k];
25          a[k] = a[i];
26          a[i] = t;
27          i = i + 1;
28      }
29  }
30
31  void main(void) {
32      int i;
33      i = 0;
34      while(i < 10) {
35          x[i] = input();
36          i = i + 1;
37      }
38      sort(x, 0, 10);
39      i = 0;
40      while(i < 10) {
41          output(x[i]);
42          i = i + 1;
43      }
44  }
45
```

Fonte: (LOUDEN, 2004).

Tabela 15 – Correspondência entre códigos intermediário, assembly e binário gerados para o programa de ordenação de vetor.

Quádrupla	Assembly	Binário
	NOP	11101100000000000000000000000000

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
	MOV R0, #0	11100010101100000000000000000000
	MOV R29, #0	11100010101100000111010000000000
	MOV R30, #0	11100010101100000111100000000000
	MOV R31, #0	11100010101100000111110000000000
(ALLOC, -, x, 10)	ADD R30, R30, #10	11100010000011110111100000001010
	B main	11100100000000000000000001110011
(FUNCTION, minloc, int, -)	minloc:	
	STR R29, R30	11101010111110111010000000000000
	MOV R29, R30	11100000101100000111011111000000
	ADD R30, R30, #1	11100010000011110111100000000001
	STR R31, R30	11101010111110111110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(PARAM, minloc, a, int)		
(PARAM, minloc, low, int)		
(PARAM, minloc, high, int)		
(ALLOC, minloc, i, -)	ADD R30, R30, #1	11100010000011110111100000000001
(ALLOC, minloc, x, -)	ADD R30, R30, #1	11100010000011110111100000000001
(ALLOC, minloc, k, -)	ADD R30, R30, #1	11100010000011110111100000000001
(LOAD, low, -, t1)	SUB R2, R29, #1	11100010001011101000100000000010
	LDR R1, R2	11101000100010000010000000000000
(STORE, k, -, t1)	STR R1, [R29, #4]	11101011111101000010000000000100
(LOAD, a, -, t2)	SUB R3, R29, #3	11100010001011101000110000000011
	LDR R2, R3	11101000100011000100000000000000
(ADD, t2, t1, t3)	ADD R3, R2, R1	11100000000000010000110000100000
(LOAD, t3, -, t4)	LDR R4, R3	11101000100011001000000000000000

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(STORE, x, -,	STR R4, [R29, #3]	11101011111101001000000000000011
(ADD, t1, 1, t3)	ADD R3, R1, #1	11100010000000001000110000000001
(STORE, i, -, t3)	STR R3, [R29, #2]	11101011111101000110000000000010
(LABEL, -, -, L0)	L0: NOP	11101100000000000000000000000000
(LOAD, i, -, t1)	LDR R1, [R29, #2]	11101001111101000010000000000010
(LOAD, high, -, t2)	SUB R5, R29, #1 LDR R2, R5	11100010001011101001010000000001 11101000100101000100000000000000
(LT, t1, t2, t3)	CMP R1, R2	11100000110000001000000001000000
(IFF, t3, -, L1)	BGE L1	10100100000000000000000000101111
(LOAD, a, -, t3)	SUB R5, R29, #3 LDR R2, R5	11100010001011101001010000000011 11101000100101000110000000000000
(ADD, t3, t1, t4)	ADD R4, R3, R1	11100000000000011001000000100000
(LOAD, t4, -, t5)	LDR R5, R4	11101000100100001010000000000000
(LOAD, x, -, t4)	LDR R4, [R29, #3]	11101001111101001000000000000011
(LT, t5, t4, t5)	CMP R5, R4	11100000110000101000000010000000
(IFF, t5, -, L2)	BGE L2	10100100000000000000000000101010
(ADD, t3, t1, t4)	ADD R4, R3, R1	11100000000000011001000000100000
(LOAD, t4, -, t5)	LDR R5, R4	11101000100100001010000000000000
(STORE, x, -, t5)	STR R5, [R29, #3]	11101011111101001010000000000011
(STORE, k, -, t1)	STR R1, [R29, #4]	11101011111101000010000000000100
	L2:	

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(LABEL, -, -,	NOP	11101100000000000000000000000000
(LOAD, i, -, t1)	LDR R1, [R29, #1]	11101001111101000010000000000010
(ADD, t1, 1, t4)	ADD R4, R1, #1	11100010000000001001000000000001
(STORE, i, -, t4)	STR R4, [R29, #2]	11101011111101001000000000000010
(GOTO, -, -, L0)	B L0	11100100000000000000000000011001
(LABEL, -, -, L1)	L1: NOP	 11101100000000000000000000000000
(LOAD, k, -, t1)	LDR R1, [R29, #4]	11101001111101000010000000000100
(RETURN, t1, -, -)	MOV R0, R1	11100000101100000000000000100000
	LDR R31, [R29, #1]	11101001111101111110000000000001
	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111110000000000000000000
(END, minloc, -, -)	LDR R31, [R29, #1]	11101001111101111110000000000001
	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111110000000000000000000
(FUNCTION, sort, int, -)	sort: STR R29, R30	 11101010111110111010000000000000
	MOV R29, R30	11100000101100000111011111000000
	ADD R30, R30, #1	11100010000011110111100000000001
	STR R31, R30	11101010111110111110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(PARAM, sort, a, int)		
(PARAM, sort, low, int)		
(PARAM, sort, high, int)		
(ALLOC, sort, i, -)	ADD R30, R30, #1	11100010000011110111100000000001

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(ALLOC, sort, k, -)	ADD R30, R30, #1	11100010000011110111100000000001
(LOAD, low, -, t1)	SUB R6, R29, #2	11100010001011101001100000000010
	LDR R1, R6	11101000100110000010000000000000
(STORE, i, -, t1)	STR R1, [R29, #2]	11101011111101000010000000000010
(LABEL, -, -, L3)	L3:	
	NOP	11101100000000000000000000000000
(LOAD, i, -, t1)	LDR R1, [R29, #2]	11101001111101000010000000000010
(LOAD, high, -, t2)	SUB R6, R29, #1	11100010001011101001100000000001
	LDR R2, R6	11101000100110000100000000000000
(SUB, t2, 1, t3)	SUB R3, R2, #1	11100010001000010000110000000001
(LT, t1, t3, t3)	CMP R1, R3	11100000110000001000000001100000
(IFF, t3, -, L4)	BGE L4	10100100000000000000000001101110
(ALLOC, sort, t, -)	ADD R30, R30, #1	11100010000011110111100000000001
(PUSH, -, -, t1)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(PUSH, -, -, t2)	STR R2, R30	11101010111110000100000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(LOAD, a, -, t3)	SUB R6, R29, #3	11100010001011101001100000000011
	LDR R2, R6	11101000100110000110000000000000
(ARG, t3, -, -)	STR R3, R30	11101010111110000110000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(ARG, t1, -, -)	STR R1, R30	11101010111110000010000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(ARG, t2, -, -)	STR R2, R30	11101010111110000100000000000000
	ADD R30, R30, #1	11100010000011110111100000000001
(CALL, minloc, 3, -)	BL minloc	11100110000000000000000000000111
	SUB R30, R30, #3	11100010001011110111100000000011
	SUB R30, R30, #1	11100010001011110111100000000001

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(POP, -, -,	LDR R2, R30	11101000111110000100000000000000
(POP, -, -,	SUB R30, R30, #1	11100010001011110111100000000001
t1)	LDR R1, R30	11101000111110000100000000000000
(MOVE, t0, -, t3)	MOV R3, R0	11100000101100000000110000000000
(STORE, k, -, t3)	STR R3, [R29, #3]	11101011111101000110000000000011
(LOAD, a, -,	SUB R6, R29, #3	11100010001011101001100000000011
t4)	LDR R4, R6	11101000100110001000000000000000
(ADD, t4, t3, t5)	ADD R5, R4, R3	11100000000000100001010001100000
(LOAD, t5, -, t6)	LDR R6, R5	11101000100101001100000000000000
(STORE, t, -, t6)	STR R6, [R29, #4]	11101011111101001100000000000100
(ADD, t4, t1,	ADD R5, R4, R1	11100000000000100001010000100000
t5)		
(LOAD, t5, -, t7)	LDR R7, R5	11101000100101001110000000000000
(ADD, t4, t3, t5)	ADD R5, R4, R3	11100000000000100001010001100000
(STORE, t5, -, t7)	STR R7, R5	11101010100101001110000000000000
(ADD, t4, t1, t5)	ADD R5, R4, R1	11100000000000100001010000100000
(STORE, t5, -, t6)	STR R6, R5	11101010100101001100000000000000
(ADD, t1, 1, t5)	ADD R5, R1, #1	11100010000000001001010000000001
(STORE, i, -, t5)	STR R5, [R29, #2]	11101011111101001010000000000010
(GOTO, -, -, L3)	B L3	11100100000000000000000001000100
(LABEL, -, -, L4)	L4: NOP	11101100000000000000000000000000
	LDR R31, [R29, #1]	11101001111101111110000000000001

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(END, sort, -, -)	MOV R30, R29	11100000101100000111101110100000
	LDR R29, R30	11101000111110111010000000000000
	BX R31	11100101111110000000000000000000
(FUNCTION, main, void, -)	main: MOV R29, R30	11100000101100000111011111000000
(ALLOC, main, i, -)	ADD R30, R30, #1	11100010000011110111100000000001
(LOAD, 0, -, t1)	LDR R1, #0	11101001000000000010000000000000
(STORE, i, -, t1)	STR R1, [R29, #0]	11101011111101000010000000000000
(LABEL, -, -, L5)	L5: NOP	11101100000000000000000000000000
(LOAD, i, -, t1)	LDR R1, [R29, #0]	11101001111101000010000000000000
(LT, t1, 10, t2)	CMP R1, #10	11100010110000001000000000001010
(IFF, t2, -, L6)	BGE L6	10100100000000000000000010000100
(CALL, input, 0, t0)	IN R0 NOP	11101100100000000000000000000000 11101100000000000000000000000000
(MOVE, t0, -, t2)	MOV R2, R0	11100000101100000000100000000000
(LOAD, x, -, t3)	LDR R3, #0	11101001000000000110000000000000
(ADD, t3, t1, t4)	ADD R4, R3, R1	11100000000000011001000000100000
(STORE, t4, -, t2)	STR R2, R4	11101010100100000100000000000000
(ADD, t1, 1, t2)	ADD R2, R1, #1	11100010000000001000100000000001
(STORE, i, -, t2)	STR R2, [R29, #0]	11101011111101000100000000000000
(GOTO, -, -, L5)	B L5	11100100000000000000000001110111
	L6:	

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(LABEL, -, -,	NOP	11101100000000000000000000000000
(LOAD, x, -, t1)	LDR R1, #0	11101001000000000010000000000000
(ARG, t1, -, -)	STR R1, R30 ADD R30, R30, #1	11101010111110000010000000000000 11100010000011110111100000000001
(ARG, 0, -, -)	LDR R8, #0 STR R8, R30 ADD R30, R30, #1	11101001000000010000000000000000 11101010111110010000000000000000 11100010000011110111100000000001
(ARG, 10, -, -)	LDR R8, #10 STR R8, R30 ADD R30, R30, #1	11101001000000010000000000001010 11101010111110010000000000000000 11100010000011110111100000000001
(CALL, sort, 3, -)	BL sort SUB R30, R30, #3	11100110000000000000000000000110 11100010001011110111100000000011
(LOAD, 0, -, t1)	LDR R1, #0	11101001000000000010000000000000
(STORE, i, -, t1)	STR R1, [R29, #0]	11101011111101000010000000000000
(LABEL, -, -, L7)	L7: NOP	11101100000000000000000000000000
(LOAD, i, -, t1)	LDR R1, [R29, #0]	11101001111101000010000000000000
(LT, t1, 10, t2)	CMP R1, #10	11100010110000001000000000001010
(IFF, t2, -, L0)	BGT L8	10100100000000000000000010011111
(LOAD, x, -, t2)	LDR R2, #0	11101001000000000100000000000000
(ADD, t2, t1, t3)	ADD R3, R2, R1	11100000000000010000110000100000
(LOAD, t3, -, t4)	LDR R4, R3	11101000100011001000000000000000
(MOVE, t4, -, t0)	MOV R4, R0	11100000101100000001000000000000
(CALL, output, 0, -)	OUT R0 NOP	11101100110000000000000000000000 11101100000000000000000000000000

continua na próxima página

continuação da Tabela 15

Quádrupla	Assembly	Binário
(ADD, t1, 1,	ADD R3, R1, #1	11100010000000001000110000000001
(STORE, i, -, t3)	STR R3, [R29, #0]	11101011111101000110000000000000
(GOTO, -, -, L7)	B L7	11100100000000000000000001110111
(LABEL, -, -, L8)	L8: NOP	11101100000000000000000000000000
(END, main, -, -)		
(HALT, -, -, -)	HLT	11101100010000000000000000000000

Fonte: A autora (2026).

6 Conclusão

O desenvolvimento do compilador evidenciou diversos desafios inerentes às etapas de tradução e geração de código para uma arquitetura de processador específica. A principal dificuldade encontrada esteve relacionada à implementação das chamadas de função durante a geração de código intermediário. Inicialmente, verificou-se que a troca de contexto entre funções poderia ocasionar a sobrescrita do conteúdo de registradores ainda necessários após o retorno da execução. Esse problema exigiu a implementação de um mecanismo de preservação e restauração do contexto, garantindo a integridade dos dados armazenados nos registradores ao longo das chamadas de função. A resolução dessa limitação representou um importante avanço para a confiabilidade do código gerado e proporcionou uma compreensão mais aprofundada da interação entre compiladores e arquitetura de computadores.

Como principal destaque, o compilador desenvolvido foi capaz de gerar corretamente código de máquina compatível com o conjunto de instruções do processador ARM de 32 bits projetado, demonstrando a integração bem-sucedida entre as diferentes etapas do processo de compilação. A validação experimental foi realizada por meio da execução dos algoritmos de fatorial recursivo, máximo divisor comum e *sort* diretamente na placa FPGA, todos executados conforme o comportamento esperado. Esses resultados confirmam tanto a consistência da implementação quanto a compatibilidade entre o compilador e a arquitetura desenvolvida.

Como continuidade deste trabalho, espera-se ampliar os conhecimentos adquiridos por meio do desenvolvimento dos próximos projetos previstos na sequência de laboratórios do curso, envolvendo sistemas operacionais e comunicação digital. A integração desses componentes possibilitará uma compreensão cada vez mais abrangente do funcionamento de sistemas computacionais, desde o *hardware* até as camadas de *software* responsáveis por sua operação, contribuindo para a consolidação dos conceitos estudados ao longo da formação.

Referências

LOUDEN, K. C. *Compiladores: princípios e práticas*. São Paulo: Cengage Learning, 2004. Citado 8 vezes nas páginas 6, 15, 20, 22, 23, 34, 37 e 41.

PARK, V. F. *Projeto e Implementação de Processador ARM em Verilog e Sistema de Interface com FPGA*. São José dos Campos, 2025. Disponível em: Relatório Final da UC Laboratório de Sistemas Computacionas: Arquitetura e Organização de Computadores. Citado 7 vezes nas páginas 6, 7, 8, 11, 12, 13 e 14.